

KAZAKH BRITISH TECHNICAL UNIVERSITY

SCHOOL OF IT AND ENGINEERING

Backend for Highloaded Environment

Lecture 1 — Introduction to highload and types of scaling

Fall 2026

3 credits · 15 weeks

Nurov Arnur

INSTRUCTOR

Nurov Arnur

6+

years in software development

19

age at first team lead role, at the
largest telecom operator

Now

Head of Development at Freedom
Travel

Everything in this course is material I have had to use, break, measure or defend on a real system. Office hours are for exactly that kind of conversation.

SHOW OF HANDS

Where You Are Starting From

01 Written a Spring Boot service

02 Read an EXPLAIN ANALYZE plan

03 Run a load test on your own code

04 Seen a p99 latency chart

05 Used Kafka or another broker

06 Deployed something to Kubernetes

The course is an elective with a mixed audience. Weeks 1 and 2 include levelling material on measurement tooling and the network stack.

COURSE AIMS

What This Course Trains

Engineering judgement: choosing a protocol, a storage model and a scaling strategy from requirements for latency, throughput, consistency and cost — and defending that choice with evidence.

IT IS ABOUT

Protocols, indexes, transactions, MVCC, replication, sharding, caching, queues, distributed algorithms, resilience — and how each one is measured in production.

IT IS NOT ABOUT

Any single framework. Java 21, Spring Boot, PostgreSQL, Redis, Kafka, Elasticsearch and Docker are the instruments we measure with.

COURSE CALENDAR

Fifteen Weeks

WEEKS 1–5 · FOUNDATIONS

- 1 · Highload and scaling types
- 2 · Networking and protocols I
- 3 · Networking and protocols II
- 4 · Concurrency on the JVM
- 5 · Caching

WEEKS 6–10 · DATA

- 6 · Databases I — indexes, MVCC
- 7 · Databases II — replication, sharding
- 8 · NoSQL, CAP and PACELC
- 9 · Asynchronous architecture, Kafka
- 10 · Distributed systems foundations

WEEKS 11–15 · OPERATIONS

- 11 · Scaling, resilience and the edge
- 12 · Observability and JVM performance
- 13 · Kubernetes I
- 14 · Kubernetes for highload II
- 15 · Storage, search and synthesis

30 hours of lectures, 15 hours of practice, oral ticket-based exam in weeks 16–17.

FORMAT

The Weekly Rhythm

2 HOURS

Lecture

Mechanism, trade-offs, production incidents, architectural cases.



1 HOUR

Practice

Implement in Java, then load-test, profile or read the query plan.



DELIVERABLE

Measurement

A task is accepted only with a working result and numbers, not code alone.

Work is submitted by the end of the week. Late submissions receive 50 % of the task value.

ASSESSMENT

How the Grade Is Built

Attendance / participation	10 %
Practice, weekly	15 %
SIS — four assignments	20 %
Midterm / End-term	10 %
Quizzes, weeks 4 and 12	5 %
Final exam — oral, ticket-based	40 %

60 %

is earned during the semester, week by week.

Attendance is recorded every session. An absence without a valid reason forfeits both the attendance points and the practice points for that week — there is no make-up task.

INDEPENDENT STUDY · 20 %

Four Cumulative Assignments

WEEK 6 · 5 PTS

System design

Capacity estimates, API contract, data model, caching strategy, failure modes, diagrams.

WEEK 8 · 5 PTS

Protocol implementation

gRPC, HTTP/2 or WebSocket with idempotency, timeouts and retries, measured against REST/JSON.

WEEK 13 · 5 PTS

Database tuning

Schema and indexes, query plans, partitioning, read replicas, pooling, before/after p95 and p99.

WEEK 14 · 5 PTS

Scale-out and resilience

Caching, Kafka, resilience patterns, Kubernetes with HPA, load test, bottleneck analysis.

Non-negotiable Work that cannot be explained by its author is not accepted, whatever its quality.

WEEKS 16–17 · 40 POINTS

The Final Exam

Oral and ticket-based. You draw a ticket, prepare for 20 minutes and answer for 15–20 minutes before the examination board.

10

Theory question 1

Drawn from the whole course.

10

Theory question 2

Scaling, protocols, databases, distributed systems, Kubernetes, observability.

20

System Design case

Sketched on the board and defended under follow-up questions.

Case scoring: design correctness 8 · quantitative justification 6 · failure modes and scaling 4 · answers to follow-ups 2. Ticket bank published in MS Teams by week 12.

ACADEMIC POLICY

Generative AI — Level D

Permitted and encouraged at every stage of the practical work: code and configuration, tests and load scenarios, debugging, explanation, research. This course is about how architectural decisions are made, not how fast boilerplate is typed.

ALLOWED, WITH CONDITIONS

You own the result and must understand every line of it.

Each README states in 2–3 sentences where and how AI was used.

NO ASSISTANCE AT ALL

Oral SIS defenses, quizzes, attestations, final exam.

Reliance on AI is not an excuse for factual errors, fabricated sources or code that does not work.

If you cannot explain the logic, the trade-offs or the behaviour of your own solution when questioned, that work receives 0 points.

ATTENTION

Rules That Fail You

> 30 % of lessons missed — grade F for the course.

≤ 29.4 points across the two attestations — not admitted to the final exam, grade F.

≤ 9.4 points on the final exam — grade F regardless of semester points.

Absent or late for the final exam without a plausible reason — grade F.

Maximum within attendance, participation, practice, SIS and quiz for each attestation is 30 points.

LECTURE 1 · PART ONE OF FOUR

01

Load and Its Vocabulary

Latency and throughput

Percentiles

Tail latency

SLA, SLO, SLI

DEFINITION

When a System Becomes "Highload"

Not at a particular number of requests per second. At the point where the cheapest fix — a bigger machine — stops working, and every further increase in load has to be paid for with a design decision.

Load has structure

200 RPS of analytical queries can be heavier than 20 000 RPS of cache hits.

Load has shape

Average traffic sizes nothing. Peaks, spikes and retry waves size the cluster.

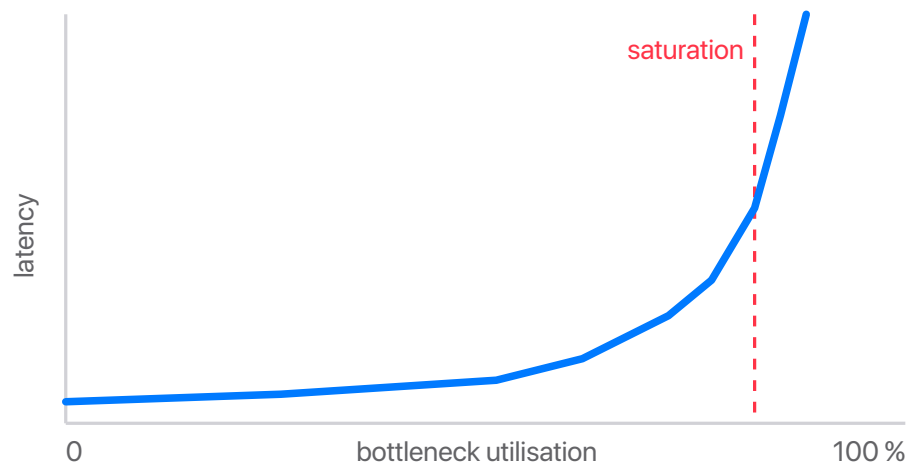
Load has a budget

A design is only correct relative to a target latency, a data volume and a cost ceiling.

So the first engineering act is never "pick a technology". It is "state the requirement in numbers".

TWO METRICS, ONE CURVE

Latency and Throughput



Throughput can still be rising while latency has already become unacceptable. The two are not interchangeable.

Latency

Time for one request. Measured in ms, reported as a percentile. What the user feels.

Throughput

Requests completed per second. What the invoice reflects.

Little's Law

$$L = \lambda \times W$$

Concurrency = arrival rate $\times$ time in system. 2 000 RPS at 50 ms means 100 requests in flight.

ORDER OF MAGNITUDE

Numbers Worth Memorising

L1 cache reference	1 ns
--------------------	------

Mutex lock / unlock	≈ 17 ns
---------------------	---------

Main memory reference	≈ 100 ns
-----------------------	----------

Compress 1 KB	≈ 2 μs
---------------	--------

Read 1 MB sequentially from RAM	≈ 3 μs
---------------------------------	--------

SSD random read	≈ 16 μs
-----------------	---------

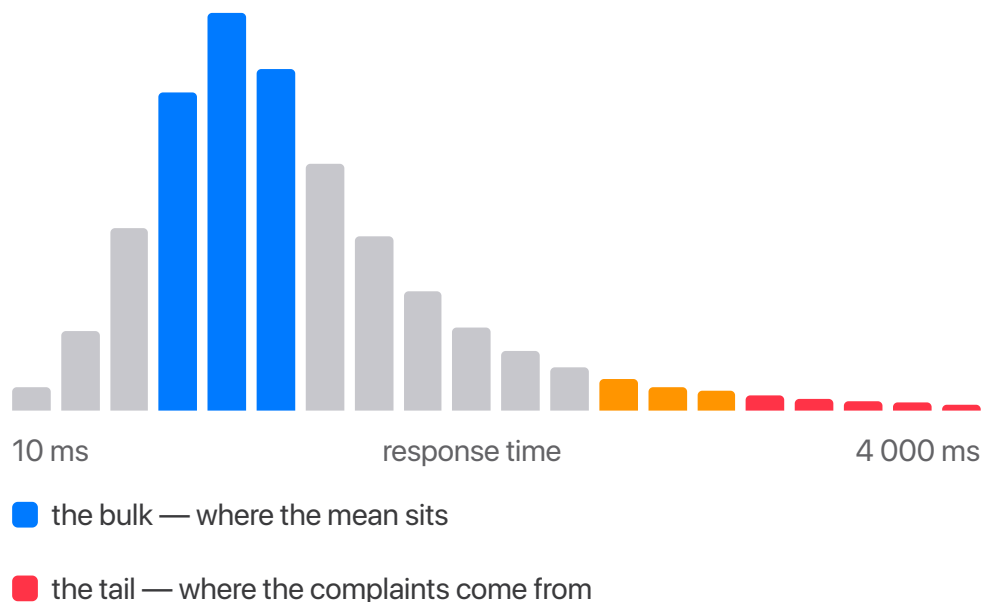
Round trip inside one datacentre	≈ 0.5 ms
----------------------------------	----------

Almaty ↔ Frankfurt round trip	≈ 100 ms
-------------------------------	----------

Source: the widely used “latency numbers every programmer should know” set, after Jeff Dean. Treat them as ratios, not measurements — you will produce your own in practice today.

PERCENTILES

Why the Average Lies



mean 40 ms

p50 — median 28 ms

p95 310 ms

p99 1 900 ms

At 2 000 RPS, p99 is 20 requests every second. Those users are real, and they are usually your heaviest ones.

TAIL LATENCY AMPLIFICATION

Tail Latency Under Fan-Out

One user request fans out to 100 backend calls. Each backend is healthy: its p99 is 100 ms. How often is the user request slow?

ALL 100 FAST

0.99^{100}

37 %

AT LEAST ONE SLOW

$1 - 0.99^{100}$

63 %

FAN-OUT TO 500

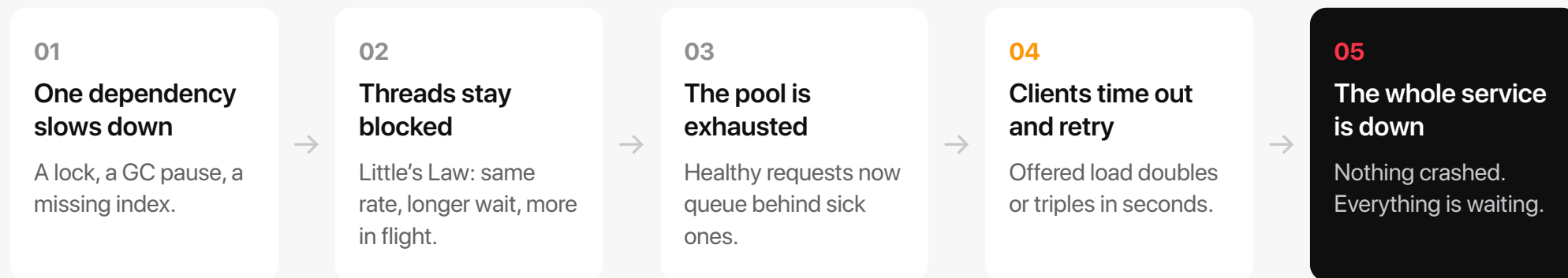
$1 - 0.99^{500}$

99 %

The user's p50 is now governed by every backend's p99. This is the core result of Dean & Barroso, *The Tail at Scale* (2013), and the reason a distributed system needs hedged requests, timeouts and budgets rather than a faster average.

FAILURE MODE

How Slowness Becomes an Outage



Weeks 11 and 12 are dedicated to breaking this chain: timeouts, bulkheads, circuit breakers, retry with backoff and jitter, graceful degradation.

The pattern is documented as the retry storm and cascading failure in Nygard, *Release It!* — required reading for week 11.

NON-FUNCTIONAL REQUIREMENTS

SLA, SLO and SLI

SLI — INDICATOR

What you measure

A number produced by the system itself.

share of requests to /search
served in < 300 ms

SLO — OBJECTIVE

What you commit to internally

A target on an SLI, chosen deliberately.

99.9 % of requests under 300
ms over 30 days

SLA — AGREEMENT

What you owe the customer

A contract with a penalty attached.

below 99.5 % monthly → 10 %
fee credit

The SLA is always looser than the SLO. If you promise the customer exactly what you promise yourself, you have no room to be wrong.

AVAILABILITY ARITHMETIC

The Error Budget

AVAILABILITY	PER YEAR	PER MONTH	WHAT IT DEMANDS
99 %	3.65 days	7.2 hours	One machine, manual restarts.
99.9 %	8.8 hours	43.8 minutes	Redundant instances, health checks, rolling deploys.
99.99 %	52.6 minutes	4.4 minutes	Multi-zone, automated failover, no human in the loop.
99.999 %	5.3 minutes	26 seconds	Multi-region, and a budget most products cannot justify.

The unused part of the budget is permission to deploy, migrate and experiment. A team that has spent its budget stops shipping features and fixes reliability instead — that is the mechanism, not a punishment.

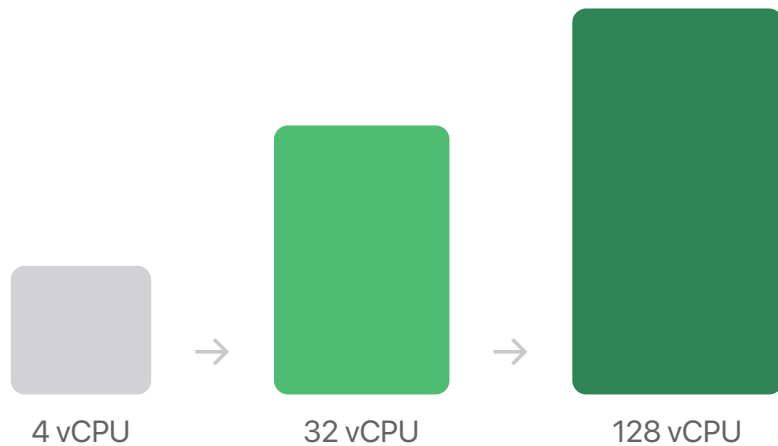
02

Types of Scaling

Up and out Stateless and stateful Reads and writes Amdahl and the USL

VERTICAL SCALING

Scaling Up



Why it is the right first answer

No code changes. No distributed state, no rebalancing, no consistency trade-offs. One machine to reason about.

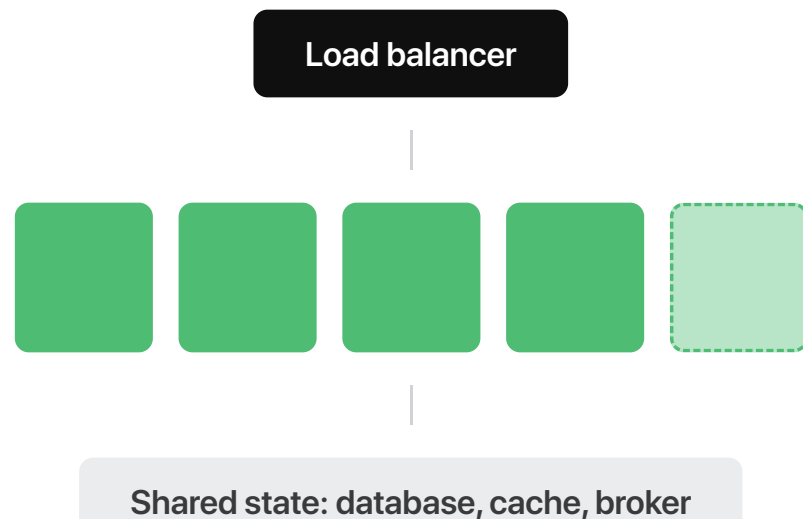
Where it stops

Price grows faster than capacity. There is a largest instance. And a single machine is a single failure domain — it also means downtime on every reboot.

Vertical scaling buys time. It does not buy availability.

HORIZONTAL SCALING

Scaling Out



What you gain

Capacity in near-linear increments, redundancy, rolling deploys, and the ability to fail one node without failing the service.

What you pay

Coordination. Every node needs the same view of state, so the bottleneck moves down the stack — to the database, the cache and the network.

Scaling out does not remove the bottleneck. It relocates it somewhere you have more options.

COMPARISON

Up or Out

DIMENSION	VERTICAL — SCALE UP	HORIZONTAL — SCALE OUT
Ceiling	The largest instance available	Coordination overhead, not hardware
Cost curve	Superlinear at the top end	Roughly linear, plus a fixed platform cost
Failure domain	One machine — reboot means downtime	N machines — lose one, keep serving
Code impact	None	Statelessness, idempotency, distributed data
Elasticity	Minutes to hours, usually with a restart	Seconds, automatable — this is what HPA does
Use it when	State is hard to split: the primary database	Work is independent: the API and worker tiers

THE LOAD-BEARING DISTINCTION

Stateless and Stateful Services

STATELESS

Any instance can serve any request

All context arrives with the request or is fetched from shared storage. Instances are interchangeable and disposable.

Scale by adding replicas behind a balancer.

Restart, replace and autoscale freely.

API tier, workers, renderers.

STATEFUL

A specific instance owns specific data

Identity matters: a replica has a role, a shard has a key range, a socket has a session.

Scale by partitioning or replicating the data.

Restarts need drain, handover and warm-up.

Databases, Kafka brokers, WebSocket gateways.

The standard architecture is not one or the other: push state down into a small number of deliberately stateful components, and keep everything above them stateless.

ACCIDENTAL STATE

Where State Hides

In-memory sessions

Works on one node. Logs users out at random on two.

Local caches

Each instance believes something different. Invalidation is now a distributed problem.

Files on local disk

Uploads land on one pod and disappear when it is rescheduled.

Scheduled jobs and timers

Ten replicas run the same nightly job ten times.

Sticky sessions

A workaround that turns your balancer into part of your data model.

The test

Kill any instance mid-request. If a user notices something other than one retry, you have state.

ASYMMETRY

Scaling Reads, Scaling Writes

READS — MANY KNOWN ANSWERS

Copy the data closer to the reader: caches, read replicas, denormalised views, CDN.

Cheap: copies do not conflict with each other.

The bill is paid in staleness — replication lag and TTL.

Weeks 5 and 7.

WRITES — FEW, ALL EXPENSIVE

Split the data by key, batch it, make it asynchronous, or narrow what needs to be consistent.

Expensive: every copy must agree on order.

The bill is paid in complexity — sharding, queues, sagas.

Weeks 7, 9 and 10.

So the read-to-write ratio is one of the first numbers to establish in any design. A 100:1 system and a 1:1 system are different problems with different answers.

TWO WAYS TO CUT A SYSTEM

Functional Decomposition and Sharding

BY FUNCTION — SPLIT THE CODE

Search

Booking

Payments

Notify

Each part gets its own resources, its own release cycle and its own scaling profile. Search can run 40 replicas while payments runs 4.

The cost is network calls between parts, and the fan-out problem from part one.

BY DATA — SPLIT THE ROWS

A–F

G–L

M–S

T–Z

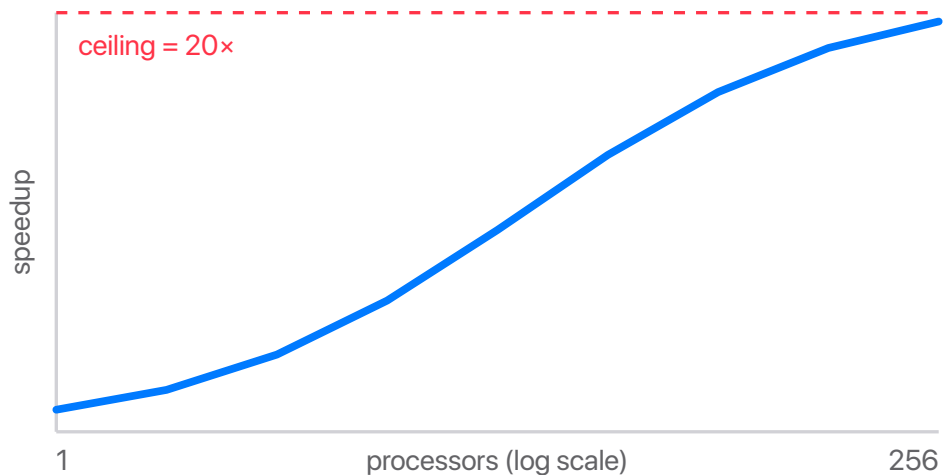
The same code runs everywhere; each instance owns a key range. This is the only way to scale writes past one machine.

The cost is cross-shard queries, hot keys, rebalancing and the choice of shard key — which is nearly irreversible.

Sequence in practice: measure → cache → replicate reads → decompose by function → shard. Sharding last, because it is the hardest step to undo.

THE FIRST CEILING

Amdahl's Law



THE IDEA

$$S(N) = 1 / ((1-p) + p/N)$$

p is the share of work that can run in parallel. The serial remainder never gets faster.

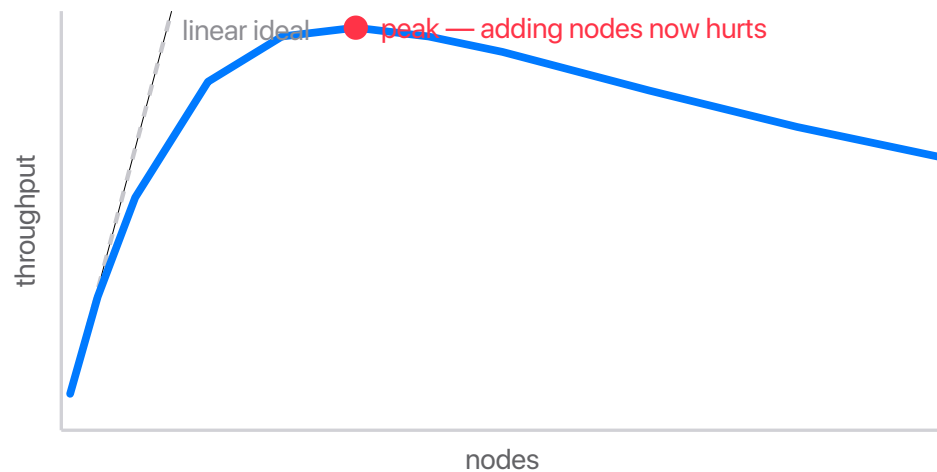
95 % parallel, 5 % serial

Maximum speedup is 20x. At 256 processors you have reached 18.6x — the last 200 processors bought almost nothing.

In a web service the serial fraction is usually a shared lock, a single-writer database or a sequence generator.

THE SECOND CEILING

The Universal Scalability Law



THE IDEA

$$C(N) = N / (1 + \alpha(N-1) + \beta N(N-1))$$

α is contention for shared resources. β is coherency — the cost of keeping nodes in agreement.

Why the curve turns down

Contention flattens throughput. Coherency reverses it: the βN^2 term grows faster than the capacity you add.

Practical consequence: capacity is something you measure and locate a peak in, not something you assume is linear.

LECTURE 1 · PART THREE OF FOUR

03

Back-of-the-Envelope Estimation

The recipe Worked example Your turn Debrief

METHOD

The Estimation Recipe

- 1 Traffic** Users $\times$ actions per user $\div$ 86 400 s $\rightarrow$ average RPS, split into reads and writes.
- 2 Peak** Average $\times$ peak factor. Use 2–3 $\times$ for daily rhythm, far more for scheduled events.
- 3 Data volume** Bytes per record $\times$ records per day $\times$ retention, plus index and replication overhead.
- 4 Working set** How much of the data is hot? That size decides whether a cache is viable.
- 5 Capacity and cost** Peak RPS $\div$ per-instance capacity, then add headroom for failure and for growth.

Round aggressively and keep the units visible. The purpose is not a precise answer; it is knowing which order of magnitude you are designing for, and therefore which problem you actually have.

WORKED EXAMPLE

A URL Shortener

Given requirements. Everything else in the design has to be derived from these five lines.

Daily active users

10 M

Redirects per user per day

5

New links per day

1 M

Retention

5 years

Latency budget, p99

50 ms

Availability target

99.95 %

STEPS 1 AND 2

Traffic and Peak

READS — REDIRECTS

$10\text{ M} \times 5 = 50\text{ M} / \text{day}$

$50\text{ M} \div 86\,400 \approx 580\text{ RPS average}$

$\times 3\text{ peak factor} \approx 1\,700\text{ RPS peak}$

WRITES — NEW LINKS

$1\text{ M} \div 86\,400 \approx 12\text{ RPS average}$

$\times 3\text{ peak factor} \approx 35\text{ RPS peak}$

$\text{bandwidth} \approx 1\,700 \times 500\text{ B} \approx 0.9\text{ MB/s}$

50:1

Read-to-write ratio

A read-dominated system. The design should spend its effort on caching and read availability, not on write throughput.

STEPS 3 AND 4

Storage and Working Set

DATA VOLUME

record ≈ 500 B with index overhead

1 M/day $\times 500$ B = 500 MB/day

$\times 365 \approx 180$ GB/year

$\times 5$ years ≈ 0.9 TB total

WORKING SET

assume 80 % of reads hit 20 % of links

hot set ≈ 10 M links $\times 500$ B ≈ 5 GB

→ fits in one Redis instance

→ at 95 % hit ratio, DB sees ~ 85 RPS

0.9 TB

five years of data — one node, no sharding needed

5 GB

hot set — comfortably in memory

3

app instances: 2 for 1 700 RPS peak, 1 for failure headroom

STEP 5

The Verdict

Ten million users, and the answer is three application instances, one PostgreSQL primary with a replica, and one Redis.

NOT THE PROBLEM

Compute. Sharding. Write throughput. A microservice split. Anything that a whiteboard instinct would have added first.

THE ACTUAL PROBLEM

Availability at 99.95 %, cache behaviour on a miss, hot-key traffic, and what happens to redirects when the cache restarts empty.

The estimate did not produce the architecture. It told us which questions the architecture has to answer.

IN PAIRS · 7 MINUTES

Your Turn — Registration Day

Course registration opens at 09:00. Estimate the capacity the system needs.

GIVEN

5 000 students, all registering the same morning.

Roughly 60 % of them are in the system within the first minute.

About 20 requests per student: browse, filter, add, confirm.

Seats per course are limited and must never be oversold.

ANSWER THESE FOUR

- 1 · Peak RPS in the first minute.
- 2 · Ratio of that peak to the daily average.
- 3 · Total data written that morning.
- 4 · What actually limits the system — and it is not RPS.

DEBRIEF

What the Numbers Show

THE ARITHMETIC

$3\,000 \text{ students} \times 20 \text{ req} = 60\,000 \text{ req}$
 $\div 60 \text{ s} \approx 1\,000 \text{ RPS}$ in the first minute
daily average $\approx 100\,000 \div 86\,400 \approx 1.2$
RPS

data written $\approx 5\,000 \times 5 \text{ rows} \times 200 \text{ B} \approx 5$
MB

$\approx 800\times$

peak-to-average ratio. Sizing this system on its average load would have been wrong by nearly three orders of magnitude.

The data is trivial. The contention is not.

5 MB is nothing. But 200 students hitting the same seat counter must be serialised, and no number of application replicas makes a single row faster.

That serialised counter is Amdahl's serial fraction, in production. Week 6 covers the locking behaviour behind it; week 10 covers the alternatives — queueing the writes and reserving seats idempotently.

LECTURE 1 · PART FOUR OF FOUR

04

This Week's Practice

The task Acceptance criteria Setup Reading

WEEK 1 PRACTICE · 1 POINT

Baseline Service

PART 1

Measure

Build a minimal Spring Boot service and report p50, p95 and p99 under a fixed load. Note the load generator, the duration and the concurrency.

PART 2

Estimate

Produce a capacity estimate for a target RPS using today's five-step recipe, then check it against your measurement.

PART 3

Compare

One large instance against several small ones behind a balancer, at equal total CPU. Explain the difference you measure.

Submit by the end of the week: a repository and a short report. Late submissions receive 50 % of the value.

ACCEPTANCE CRITERIA

What Counts as “Measured”

Reproducible

A command in the README that produces the same numbers on my machine.

Conditions stated

Hardware, JVM version and flags, concurrency, duration, warm-up.

Explained

A sentence on why the number is what it is, naming the bottleneck.

Percentiles, not averages

p50, p95 and p99. A mean on its own is not an answer.

One variable at a time

A comparison where two things changed proves nothing.

AI disclosed

Two or three sentences in the README on where and how you used it.

BEFORE NEXT WEEK

Install and Verify

JDK 21

`java -version`

Docker + Compose

`docker compose version`

k6 or wrk

`k6 version`

PostgreSQL 16 in Docker

`psql -c 'select 1'`

Git and a repository host

`git --version`

MS Teams

code announced in class

Week 2 practice is a load test of HTTP/1.1 against HTTP/2 and HTTP/3. It assumes a working toolchain, so verify each command before you arrive.

BEFORE WEEK 2

Reading and Contact

READ

Kleppmann, *Designing Data-Intensive Applications* — chapter 1. Load parameters, percentiles, scaling.

Xu, *System Design Interview* vol. 1 — back-of-the-envelope estimation.

Optional: Dean & Barroso, *The Tail at Scale* (2013). Eight pages, and the source of the 63 % result.

REACH ME

Office hours — according to the schedule.

MS Teams — course channel, announced in class.

Bring a design you are unsure about. That is the most useful thing you can do with office hours.

Questions.